

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – Coding Challenge Task

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO4: Articulation (BL4, BL5)	Assessment:	Assessment Tool 1 – Coding Challenge Task
Weightage:	40% of CIA	Total Marks:	100
CO4 Statement:	Solve optimization problems using dynamic programming and greedy strategies.		
Student Name:	Paleru Dharani Govardhan	Reg. No.:	192525280
Section / Batch:	2025/SLOT C	Date:	20-08-2026

Instructions to Students

- Answer the following question. Total Marks: 100.
- Carefully analyze the given questions.
- Write clear code for the given problem.
- Analyze the Time complexity and Space complexity of your code using dynamic programming and greedy strategies

Question Type	Focus Area	Questions
Coding Challenge Task	Focus on Code and analyze optimization problems using dynamic programming and greedy strategies	Q1

SECTION A – Coding Challenge Task

A building has multiple floors, and a limited number of eggs are available to determine the highest floor from which

an egg can be dropped without breaking. The goal is to minimize the number of attempts required in the worst case.

Use Dynamic Programming to find the optimal strategy.

Input Format:

eggs = number of eggs

floors = number of floors

Sample Input:

eggs = 2

floors = 10

Expected Output:

Min Attempts = 4

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student:

RUBRICS FOR CALCULATION

Coding Challenge Task

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly understands problem constraints, edge cases, and competitive requirements	Understands problem with minor gaps in constraints	Partial understanding; misses key constraints	Misinterprets problem or constraints
Algorithm	25%	Selects optimal algorithm with best time and space complexity for competitive constraints	Uses efficient algorithm with minor scope for improvement	Uses basic/inefficient approach	No clear algorithmic approach
Code Implementation	20%	Writes correct, efficient, and clean code that passes all constraints	Code works with minor errors or inefficiencies	Partially correct code; fails for some cases	Code does not execute or gives incorrect results
Competitive Performance	20%	Solves problem within time limits and handles large inputs effectively	Solves within acceptable time with minor delays	Struggles with time limits or larger inputs	Unable to solve within time constraints
Test Case Handling	15%	Handles all test cases including edge and hidden cases	Handles most test cases	Misses important edge cases	Fails on multiple test cases

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%				
Algorithm	25%				
Code Implementation	20%				
Competitive Performance	20%				
Test Case Handling	15%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name: _____	Name: _____	Name: _____
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Q26 – Egg Dropping Problem

Name: Paleru Dharani Govardhan

Register No.: 192525280

Question: Q26 – Egg Dropping Problem

AIM

To implement the **Egg Dropping Problem using Dynamic Programming** and determine the minimum number of attempts required in the worst case to find the highest floor from which an egg can be dropped without breaking.

OBJECTIVE

- To understand the Egg Dropping Problem.
- To apply the **Dynamic Programming** technique.
- To find the optimal strategy for dropping eggs.
- To minimize the number of attempts in the worst case.

PROBLEM STATEMENT

A building has multiple floors and a limited number of eggs. The objective is to determine the highest floor from which an egg can be dropped without breaking, while minimizing the number of attempts required in the worst case.

For example:

- **Number of eggs:** 2
- **Number of floors:** 10
- **Minimum attempts:** 4

ALGORITHM

1. Create a DP table $dp[eggs][floors]$.
2. Initialize the base cases:
 - 0 floors $\rightarrow$ 0 attempts.
 - 1 floor $\rightarrow$ 1 attempt.
 - 1 egg $\rightarrow$ number of floors attempts.
3. Consider every possible floor as the first dropping floor.
4. If the egg breaks, check the floors below.
5. If the egg does not break, check the floors above.
6. Take the maximum of both cases because we need the **worst-case** result.
7. Add 1 for the current attempt.
8. Choose the minimum among all possible dropping floors.
9. Display the minimum number of attempts.

CODE

```
def egg_drop(eggs, floors):
    dp = [[0] * (floors + 1) for _ in range(eggs + 1)]

    # Base cases
    for e in range(1, eggs + 1):
        dp[e][0] = 0
        dp[e][1] = 1

    for f in range(1, floors + 1):
        dp[1][f] = f

    # Dynamic Programming
    for e in range(2, eggs + 1):
        for f in range(2, floors + 1):
            dp[e][f] = float('inf')

            for x in range(1, f + 1):
                attempts = 1 + max(
                    dp[e - 1][x - 1],
                    dp[e][f - x]
                )

            dp[e][f] = min(dp[e][f], attempts)

    return dp[eggs][floors]

eggs = int(input())
floors = int(input())

print("Min Attempts =", egg_drop(eggs, floors))
```

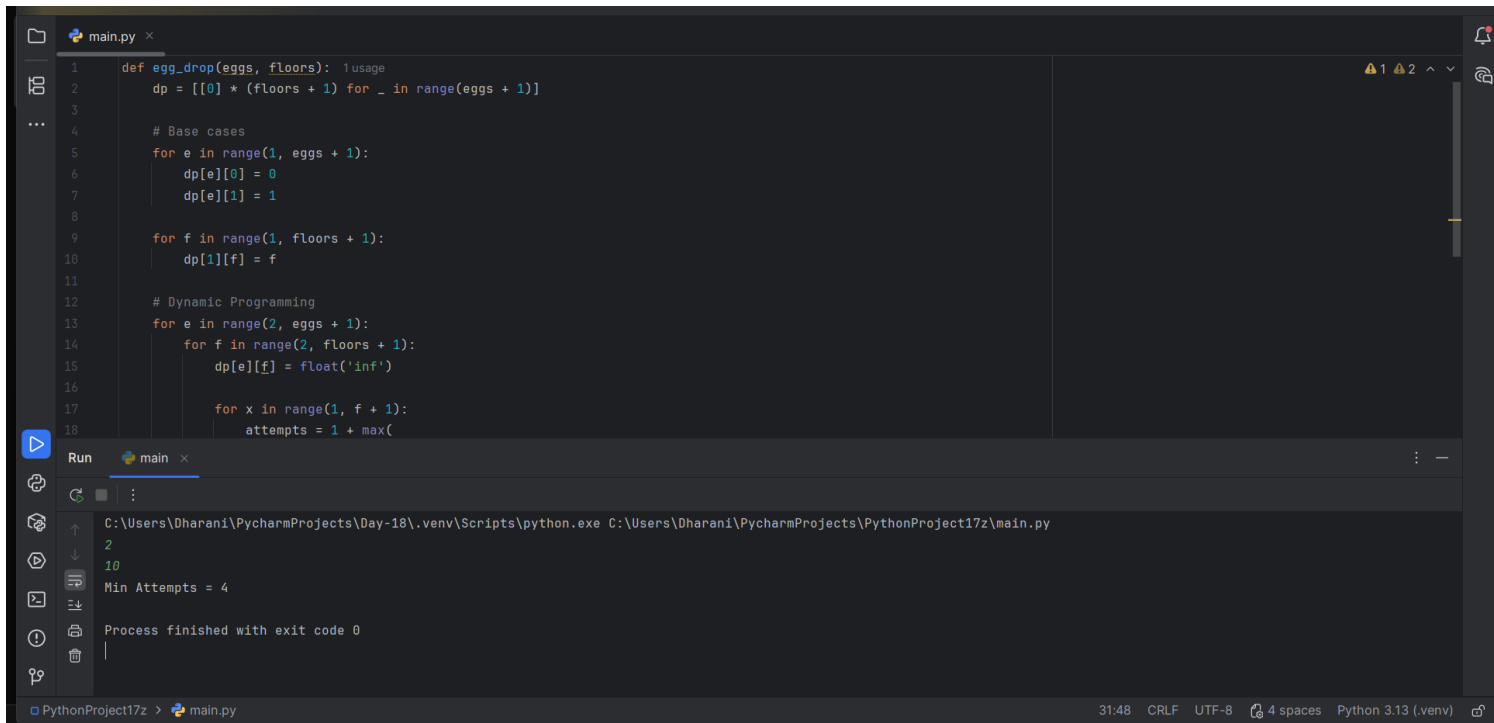
INPUT

Eggs = 2

Floors = 10

OUTPUT

Min Attempts = 4



The screenshot shows a PyCharm IDE with a Python script named `main.py` and its execution output. The script implements a dynamic programming solution for the Egg Dropping Problem. It defines a function `egg_drop(eggs, floors)` that returns the minimum number of attempts required. The script uses a 2D array `dp` to store the results of subproblems. The base cases are `dp[e][0] = 0` and `dp[e][1] = 1`. The dynamic programming part iterates over the number of eggs and floors, calculating the minimum attempts for each subproblem. The output of the script is displayed in the Run console, showing the input values 2 and 10, and the result Min Attempts = 4.

```
1 def egg_drop(eggs, floors):
2     dp = [[0] * (floors + 1) for _ in range(eggs + 1)]
3
4     # Base cases
5     for e in range(1, eggs + 1):
6         dp[e][0] = 0
7         dp[e][1] = 1
8
9     for f in range(1, floors + 1):
10        dp[1][f] = f
11
12    # Dynamic Programming
13    for e in range(2, eggs + 1):
14        for f in range(2, floors + 1):
15            dp[e][f] = float('inf')
16
17            for x in range(1, f + 1):
18                attempts = 1 + max(
19                    dp[x][x - 1],
20                    dp[e - x][f - x]
21                )
22                dp[e][f] = min(dp[e][f], attempts)
23
24    return dp[eggs][floors]
```

Run console output:

```
C:\Users\Dharani\PycharmProjects\Day-18\.venv\Scripts\python.exe C:\Users\Dharani\PycharmProjects\PythonProject17z\main.py
2
10
Min Attempts = 4
Process finished with exit code 0
```

TIME COMPLEXITY

$O(E \times F^2)$

Where:

- E = number of eggs
- F = number of floors

SPACE COMPLEXITY

$O(E \times F)$

CONCLUSION

Thus, the **Egg Dropping Problem** was successfully solved using **Dynamic Programming**. For **2 eggs and 10 floors**, the minimum number of attempts required in the worst case is **4**.